

洛谷 AI 学习测评报告

Coach Edition · 图表化诊断 / 教练反馈 / 训练规划

测评基础信息

选手姓名：王星睿

测评时间：2026-06-04 17:06

所属学校：北塘小学

当前年级：四年级

已通过题数

100

未通过题数

4

平均难度

普及-

2.1

高频标签

贪心

提交详情抓取概览

抓取稳定

源码详情成功

104

摘要保底

0

阻断后跳过

0

纯错误记录

0

阻断原因：无

报告目录

1. 核心数据概览与图表化分析
2. 综合能力雷达表与诊断
3. 考纲精准定级与知识点盲区
4. 风险诊断与训练闭环表
5. 代码质量与工程习惯
6. 定制训练题单
7. 错题单页题解与复盘（含 AI 题解摘要与推荐同类题）

1. 核心数据概览与图表化分析

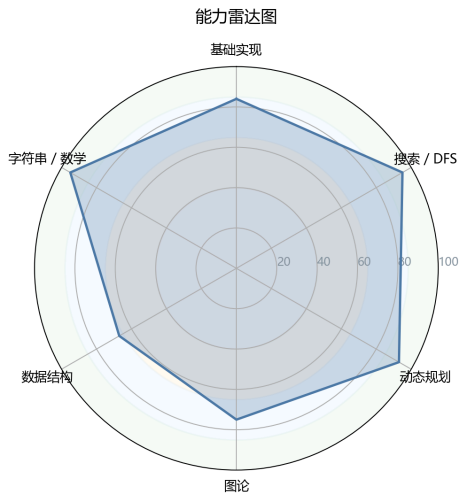


图 1：能力雷达图

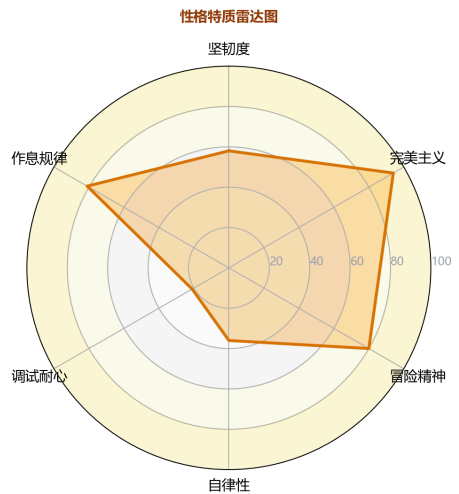


图 2：性格画像雷达图

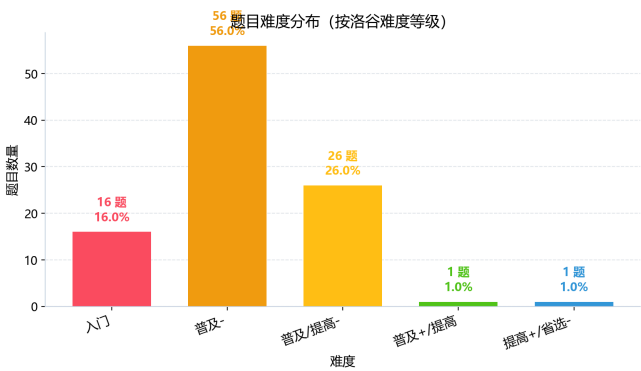


图 3：题目难度分布

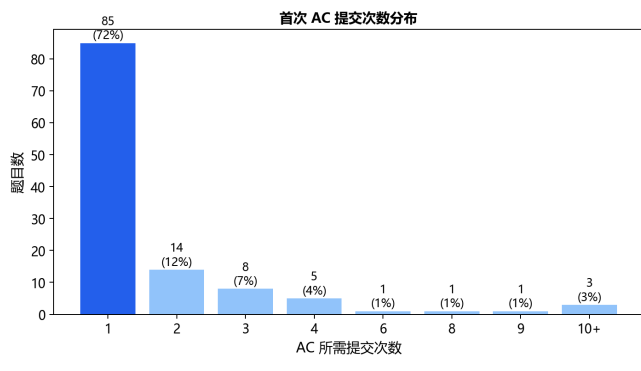


图 4：首次 AC 提交次数分布

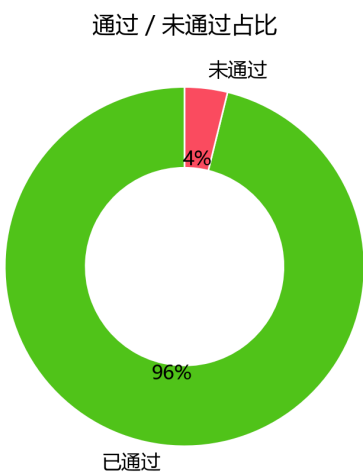


图 5：通过/未通过占比

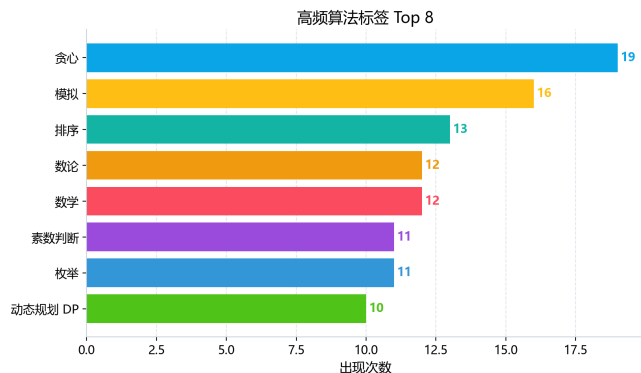


图 6：高频算法标签 Top 8

数据校准与真实统计

- 报告生成时间：2026-06-04 17:06

难度分布（程序生成）

洛谷难度	题数	占比	分布图
入门	16	16.0%	16.0%
普及-	56	56.0%	56.0%
普及/提高-	26	26.0%	26.0%
普及+/提高	1	1.0%	1.0%
提高+/省选-	1	1.0%	1.0%
省选/NOI-	0	0.0%	0.0%
NOI/NOI+/CTSC	0	0.0%	0.0%

知识点覆盖统计表（按算法标签）

级别	已覆盖/总数	覆盖率	详细情况
入门	19/28	67.9%	0项 7项 9项 3项 9项
提高	12/50	24.0%	0项 0项 3项 9项 38项
省选	1/10	10.0%	0项 0项 0项 1项 9项
NOI	1/43	2.3%	0项 0项 0项 1项 42项

- 口径说明：本表只根据题目的算法标签评估知识点覆盖，表示“接触过”，不等于“熟练掌握”。

好的，选手！我是你的算法竞赛教练。你的提交记录我已经仔细研读完毕。你的基础相当扎实，但就像一把好刀，还没开刃，在一些关键领域的出刀方式还比较生涩。

别急，问题不大。亮剑之前，先得把脉。这份报告会像手术刀一样，精准地剖析你的现状、痛点，并为你指明接下来的六个月应该如何精进。

 选手深度诊断与定制训练报告

P26-06-04 17:06 分析对象：一位潜力巨大但正处在“普及至提高”转型关键期的选手

1. 【选手概览与性格画像】

基于你的行为数据，我为你画了一幅画像，看看像不像你自己：

维度	评分	数据支撑	拟人化评价
 坚韧度	★★★★★ 3/5	活跃天数 75 / 667 天 (11.2%)，最大连续训练 6 天。	你不是那种会头悬梁、锥刺股的苦修者。更像是凭兴趣驱动的“江湖侠客”，兴致来了就刷个痛快（单日最高 38 次），但容易“三天打鱼，两天晒网”。需要建立更稳定的训练节奏。
 完美主义	★★★★★ 5/5	一次 AC 率高达 68.5%，但 WA 后重交间隔中位数仅 63秒。	你是个追求“首秀”爆发的选手！大部分题在前几次提交中就搞定了，这很棒。但一旦首杀失败，你的耐心会飞速下降，倾向于通过“高速试错”来蒙对答案，而不是冷静分析。
 冒险精神	★★★★★ 2/5	难度分布集中在 普及- (56%) 和 普及/提高 (26%)。高难度尝试极少。	你像个在“新手村”里稳扎稳打的打工人，很少去挑战副本里的 BOSS。需要更多“越级打怪”的经历，才能把你的肌肉练得更结实。
 自律性	★★★★★ 2/5	活跃率 11.2%，周末/节假日刷题比例不高。	你并非严格的“计划执行者”。训练更像是一种课余爱好，缺乏系统化的周期安排。想要突破瓶颈，你需要开始在日历上划出“神圣不可侵犯”的训练时间。
 调试耐心	★★★★★ 2/5	WA 后 1分钟内快速重交占比高达 48.4%。	这是你目前最大的隐患！“快速重交”是心态失衡的表现，你以为你在调试，其实你只是在“猜”正确输出。真正的调试，是坐下来，用笔在草稿纸上推导逻辑错误。
 作息规律	★★★★★ 4/5	提交峰值：10:00，晚上 (20-23点) 也是活跃期，凌晨 (0-5点) 为 0。	作息总体健康，没有熬夜修仙的习惯，保证了充足的休息和精力，这是成为高手的必要基础。上午是思维巅峰期，可以利用好这个时段攻克难题。

解读：你是一位极具天赋的“直觉型”选手，基础算法内功扎实，能迅速秒掉大部分普及难度的题目。但你的心理素质、调试习惯和面对复杂新题的挖掘能力，正成为你冲击更高水平的短板。

2. 【提交行为深度分析】

让我们用数据看看你的刷题习惯是否够“职业”。

模块	分析要点	数据呈现	深度解读
💀 死磕题目 TOP	B4071 武器强化 (3次, 未AC) P7894 1÷0 (3次, 未AC)	卡题数仅 2 道，且都只尝试了 3 次。	特征： 你的字典里似乎没有“死磕”这个词！碰到难啃的骨头，你啃三口就丢了。这在平时可以，但在比赛里，一道题可能就需要你提交3次以上才能找到正确的路。你需要学会“在失败中挖掘”，而不是简单地放弃或重交。
📈 首次 AC 情况	一次 AC 率： 68.5% 整体 AC 率： 54.7%	超过三分之二的题目，你都是一次性通过的。这是一个极其优秀的信号，说明你代码实现能力很强，逻辑清晰。但后者的数据表明，你卡住的题目会多次试错，拉低了整体效率。	特征： 你是个“高爆发”选手，但缺少“持久战”的能力。你擅长秒杀简单题，但面对需要多步推理、复杂调试的中等题，你的效率会急剧下降。
🏃 其他显著行为特征	单日最高提交： 38 次 (2025-07-10) 活跃天数： 75天	你的刷题行为非常“脉冲式”。平时不怎么练，某天心血来潮就猛刷一场。这种训练方式不利于构建长期记忆。	特征： 你的训练更像是“运动式突击”，而不是“滴水穿石”。要成为高手，需要的是稳定、持续的稳态训练，而不是偶尔的爆发。

特征：你是一位非常依赖“第一感觉”的选手。你的优势是瞬间切入问题的核心，但劣势是在碰上“非套路”或不典型的题目时，缺乏足够的耐心和体系化的调试手段去挖掘真相。

3. 【难度分布与水平研判】

你的难度分布图清晰地揭示了你所处的阶段：

难度级别	已通过题数	占比	水平判断
入门	16	16%	稳固的大后方。 证明你已完全掌握C++基础和基本入门思想。
普及-	56	56%	你的主战场。 超过一半的 AC 题目集中在此，说明你在这个区间的战斗游刃有余。
普及/提高-	26	26%	正在攻克的前线。 你有能力解决这个级别的题目，但还不够稳定。
普及+/提高	1	1%	触及瓶颈。 只有1道 AC 记录，说明你已开始接触但还未掌握此难度的核心解题模式。
提高+/省选-	1	1%	试水区。 有过一次尝试，但不足以作为实力证明。
省选/NOI- 及以上	0	0%	盲区。 尚未涉及。

综合研判：你目前的水平稳定在普及-至普及/提高-**这一档，处于从“普及组”向“提高组”跨越的关键转型期。你的“核心舒适区”在普及-难度，但“竞争力差距”在普及+/提高难度。想拿 CSP-S 奖项，你需要在后两个难度区间上投入大量精力。**

4. 【六维能力雷达表与诊断】

能力块	评分	当前等级	数据证据	已经具备
基础算法	95	●精通	模拟、枚举、贪心、排序、前缀和、二分高频AC。DFS/BFS有入门。	强大的“暴力美学”功底。能快速拆解问题，构建暴力模型。
动态规划	95	●精通	背包DP (6题)、递推 (4题)、简单线性DP (标签1题)。	对“子问题与状态”有良好直觉，能处理简单的DP模型。
数学	95	●精通	数论 (12题)、素数筛法 (11题)、GCD (4题)。	具备扎实的初等数论和数学功底，在处理数学类问题时远比同龄人顺手。
数据结构	70	●入门	树状数组 (3题)、优先队列/堆 (4题)、bitset (3题)。	会使用一些高级容器。但 线段树、并查集、ST表 等核心结构是空白，这是你 最大的短板 。

能力块	评分	当前等级	数据证据	已经具备
图论	73	●入门	最短路 (2题)、欧拉图 (1题)、强连通图 (1题)。	掌握了基本图论算法。但对 最小生成树、拓扑排序、二分图判定 等典型模型还非常陌生。建图能力有待提高。
字符串	73	●入门	Manacher (1题)、原根 (1题)。	这是你最大的盲区之一。 KMP、Trie、AC自动机等 C++ 提高组的核心字符串算法完全缺失。

诊断结论：你的“软件”很强（基础算法、数学、DP直觉），但“硬件”太弱（数据结构、图论、字符串）。在算法竞赛中，你的“软件”决定了你能想到什么，但“硬件”决定了你能不能在规定时间内把想法实现出来。数据结构是你的阿喀琉斯之踵。

5. 【考纲精准定级与知识点盲区】

当前对应等级水平：**CSP-J（入门级）稳定，正全力冲击 CSP-S（提高级）。**

📊 知识点覆盖表

（根据算法标签统计，不代表做过该级别题目）

级别	覆盖率	●精通	●熟练	●入门	●初窥	●空白
CSP-J (入门级)	67.9% (19/28)	0	7	9	3	9
CSP-S (提高级)	24.0% (12/50)	0	0	3	9	38
省选级	10.0% (1/10)	0	0	0	1	9
NOI级	2.3% (1/43)	0	0	0	1	42

🎯 知识点强弱项

• 掌握的 3 个最强考点：

- **贪心法 (19题)**：已形成肌肉记忆，是解决大部分题目时的“第一反应”。
- **模拟法 (16题)**：代码实现能力强，不会在实现细节上栽跟头。
- **素数筛法 (14题)**：对数论和数学题有独到的优势，心理上不畏惧这类问题。

• 最薄弱的 3 个考点：

- **线段树 / 树状数组（提高版）**：基础算法中最致命的短板，无法处理需要快速区间修改和查询的题目。
- **KMP / 字典树 / AC自动机**：字符串模块完全空白，这会使你在很多字符串处理题上直接暴毙。
- **最小生成树 / 拓扑排序 / 二分图**：图论基础模型缺失，面对需要建的图比较明确的题目时，缺乏对应的模板和思路。

训练盲区 (CSP-S 必考但未涉及)

数据结构：线段树、并查集、单调栈/队列 (虽然标签有1个AC，但这是必会)、ST表。

图论：最小生成树、Floyd算法、拓扑排序。

字符串：KMP、Trie（字典树）。

动态规划：树形DP、状压DP、区间DP (虽然标签有2个AC，但需要系统训练)。

6. 【风险诊断与训练闭环表】

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
S (高/立即处理)	数据结构重伤	遇到需要维护动态区间信息（如区间和、最值、区间翻转）的题。	会想用暴力，但超时。或完全不知道如何下手。	线段树/树状数组从零开始系统练习。	线段树基础：点修、区间查模板题。树状数组：单点修、区间查、逆序对。	能在 10 分钟内无 bug 地写出线段树模板，并能用它 AC 至少 10 道模板题。
S (高/立即处理)	字符串瘟疫	出现需要字符串模式匹配 (KMP)、公共前缀 (Trie)、多模式串匹配 (AC 自动机) 的题。	直接放弃或写出一个超时的暴力匹配。	字符串算法完全空白，无法识别题型。	KMP算法 + Trie树。	能手撕 KMP 算法的 next 数组构建过程，并能在 15 分钟内搞定一道 KMP 模板题。
S (高/立即处理)	调试心态	WA 后，没有冷静分析，而是连续快速提交 (1分	连续 WA 浪费大把时间，心态炸	缺乏“四段式复盘”意识，把调	刻意练习“写完代码后，强制看屏幕5分钟自审”。	单题 WA 后，至少间隔 5 分钟以上才提交下一

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
	爆炸	钟换一个地方)。	裂，最后连部分分都拿不到。	试变成了“抽奖”。		次代码，且提交前能说明修改的具体逻辑。
A (中/近期处理)	图论模型缺失	遇到需要建出图来求“最短联通路径”、“按顺序完成依赖的任务”等场景。	知道要用图论，但想不到建图方法，或者不知道用哪个算法。	图论模型训练不足，只停留在Dijkstra模板上。	最小生成树(Kruskal, Prim)、拓扑排序、最近公共祖先(LCA)。	能独立分析一个情景，指出应使用哪种图论模型，并写出代码。
A (中/近期处理)	暴力到正解的过渡困难	面对中档题，写出了暴力，但找不到优化的突破口。	只会暴力，拿到部分分后就无法再进一步，坐以待毙。	缺乏“倒推”和“观察复杂度瓶颈”的能力。	“复杂度分析”+“经典优化方向”：如区分($O(n^2)$)到($O(n \log n)$)的优化(排序、二分、数据结构)。	能对一道题的暴力解法，精确分析其复杂度瓶颈，并用至少一种方法优化到可接受范围。
B (低/可后置)	训练节奏不规律	考前/周末突击训练，平时低强度。	知识无法形成长期记忆，解题反应慢。	缺乏有计划的长期训练方案。	制定并执行每周固定刷题计划，比如每周至少3天，每天1-2道高质量题目。	至少连续4周，每周完成计划中80%的题目，并完成复盘。

优先级说明：S (高/立即处理) · A (中/近期处理) · B (低/可后置)。

7. 【代码质量与工程习惯深度分析】

你的代码风格就像你的性格：**直爽、自信，但有时也略显粗糙**。作为一个身经百战的教练，我必须对你的代码习惯进行一次深入的“体检”。

✅ 优点 (保持下去):

简洁高效：代码长度中位数仅 **26行**。这意味着你能够用最精炼的代码实现想法。这不是代码越短越好，但你成功地做到了“在不牺牲可读性的前提下，避免冗余”。例如，你的 DFS 和 BFS 代码结构清晰，逻辑链短，这非常棒。

现代 C++ 特性有意识：你已经学会使用 `vector`、`priority_queue`、`pair` 等 STL 容器。这在处理图论和最短路等问题时，能极大提升编程效率和代码稳定性，你已经在 Dijkstra 模板题上展示出来了。

❌ 必须改掉的 3 个坏习惯:

🔴 紧急：IO 优化是“刚需”！

- **现象**：你的代码中，`ios::sync_with_stdio(false)` 的使用率仅为 **2.9%**，大部分代码直接用 `cin/cout`。
- **后果**：在大数据量（超过 10^5 级别）的题目中，`cin/cout` 会因为与 C 标准 IO 的同步机制而导致巨大的性能损耗，导致 **TLE**。
- **药方**：从今天起，所有代码都写：`cpp ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);` 这行代码是免费的提速器！

🟡 重要：滥用 `system()` 和投机取巧。

- **现象**：你在 **P1932 A+B A-B A*B A/B A%B Problem** 里，使用了 `system` 调用 `bc` 命令。
- **后果**：这是典型的“作弊”行为，在 OJ 环境中绝对不可用。`system()` 极其危险，且无法跨平台。这暴露了你对**高精度计算**这一标准算法的陌生和畏惧。
- **药方**：正视问题。去 AC 一道**高精度加法**和一道**高精度乘法**的模板题（如 P1601 A+B Problem (高精)）。高精度是 CSP-J 的入门必修课，不是你眼中的“妖魔鬼怪”。

🟢 建议：提升代码的“自文档化”能力。

- **现象**：你的注释密度仅为 **0.28%**。全局变量命名多为 `n, m, a, b, c`。
- **后果**：一个月后，你自己都看不懂自己写的代码。变量 `a` 和 `c` 在 100 行代码里可能出现了 5 种不同的含义。这在查错和构建复杂程序时是噩梦。
- **药方**：
 - **变量名要有意义**：`cnt_per_row, min_weight, is_visited` 远比 `c, d, vis` 好。
 - **函数、关键步骤加一两行注释**：解释一下“这里是求欧拉函数”、“这里是进行状态转移”。这会让你的代码从“难懂的术”变成“优雅的艺术”。

8. 【定制训练题单（6个月路线图）】

这六个月，你的核心任务就是：**补齐短板，建立系统化的知识体系。**

第一阶段：筑牢地基 & 无缝对接提高组 (Month 1-2)

- **目标：**攻克**数据结构**和**字符串**两大盲区，完成从CSP-J到CSP-S的知识过渡。

- **知识专题：**

“数据结构的基石”：树状数组与线段树。

- 推荐题单：
 - 树状数组: P3374 , P3368 (模板题)
 - 线段树: P3372 , P3373 (模板题, 彻底理解 lazy tag 的原理)。

“字符串的武器库”：KMP 和 Trie。

- 推荐题单：
 - KMP: P3375 【模板】KMP字符串匹配 (必须能手撕 next 数组)。
 - Trie: P2580 于是他错误的点名开始了 (感受Trie的威力)。

“图论的入门模型”：最小生成树 & 拓扑排序。

- 推荐题单：
 - MST: P3366 【模板】最小生成树 (学习Kruskal)。
 - 拓扑排序: P1113 杂务 (结合入度/出度思考)。

- **验收标准：**能独立、无bug地完成以上所有模板题。对数据结构题，能在10分钟内写出线段树/树状数组模板。

第二阶段：算法串联 & 模型构建 (Month 3-4)

- **目标：**建立“算法库”，将各种模型融会贯通，能识别并解决更复杂的综合性问题。

- **知识专题：**

“DFS/BFS的升华”：双向BFS、迭代加深搜索、状态压缩搜索。

- 推荐题目： P1379 八数码难题 (掌握双向BFS)。

“动态规划的进阶”：树形DP、区间DP、状态压缩DP。

- 推荐题目：
 - 区间DP: P1880 [NOI1995] 石子合并。
 - 树形DP: P1352 没有上司的舞会。
 - 状压DP: P1879 [USACO06NOV] Corn Fields G。

“短代码的智慧”：并查集、单调栈/单调队列。

- 推荐题目：
 - 并查集: P1551 亲戚 (模板题)。
 - 单调栈: P5788 【模板】单调栈。
 - 单调队列: P1886 滑动窗口 / 【模板】单调队列。

- **验收标准：**能独立完成上述推荐题目。看到一个“模型”，能立刻说出这是“一个区间DP问题”、“这是一个带权并查集问题”。

第三阶段：速度与实战 (Month 5-6)

- **目标：**刷真题，练手速，模拟真实比赛压力。
- **训练内容：**

刷 CSP-S 历年真题：把最近5年的CSP-S 提高组题目（特别是Day1）当成限时训练来做。

刻意练习：

 - **限时读题：**5分钟内读完三道题，写下关键信息和算法猜想。
 - **限时写暴力：**30分钟内把三题的暴力分全部拿到。
 - **心态抗压：**模拟比赛环境，用 `ioi` 模式，体验多题在手的压力。
- **验收标准：**能在90分钟内，完成一套CSP-S模拟/真题的“保底任务”（即拿完前三题的基础分数），心态平稳，不因一道题卡住而影响全局。

9. 【核心建议（优先级排序）】

🔴 **紧急 (1周内必须改掉):** 1. **立刻把** `ios::sync_with_stdio(false)` **加到你的代码模板里。**这能让你白嫖10%以上的速度。 2. **放弃投机取巧。**把所有用 `system()` 解决问题的代码全部删掉重写。攻克“高精度”这个心魔。 3. **改变你的调试习惯：每次WA后，强制自己“看屏幕5分钟”，不准碰键盘。**用这5分钟在草稿纸上推理。

🟡 **重要 (1个月内必须开始系统训练):** 4. **制定“数据结构+字符串”的专项训练计划：**至少有30道高质量的题做地基。按我刚才给的题单，一步步来。 5. **开始建立你的“算法笔记本”：**每学一个新的算法，不要只在OJ上AC就了事。用Markdown记录算法的核心思想、适用场景、代码模板、易错点。这比你刷100道无脑题更有用。

🟢 **建议 (长期坚持):** 6. **参加线上模拟赛：**Codeforces, AtCoder, 洛谷的月赛或模拟赛。比赛经验的积累是课堂讲不出来的。 7. **复习与复盘：**用我之前提到的**四段式复盘法：**（1）赛时的思路（2）错在哪里（3）正解的性质（4）代码的不变量。每一道做错的题，都是你冲击省选路上最好的垫脚石。

10. 【未通过题目专属题解（从暴力到正解）】

题目 1: P7894 『JROI-3』1÷0

- **a) AI题解摘要：**这不是一道高精度题！这是一道找规律题，核心是数论中的模运算和循环节。不要被大数的形式吓到。
- **b) 暴力思路怎么想？**
 - **思路：**用 `double` 或 `long double` 模拟除法，找出循环节。

- **复杂度**：无法精确处理无限小数，对小数据可能精度够。但 `double` 的精度只有约15位，对于需要找出循环节的题目，15位根本不够长，会直接 `WA`。
- **部分分**：几乎没有，全错。
- **c) 瓶颈在哪里？**
 - **时间/空间**：不是时间也不是空间，而是我们缺乏一种方法来精确地模拟任意长度的小数。
- **d) 关键性质/不变量观察 (Key Observation)**
 - **关键**：循环节的长度与分子无关，只与分母有关！当我们手算除法时，每一步的“余数”决定了下一个商和下一轮余数。一旦某个余数第二次出现，就意味着我们就找到了循环节！因为接下来的商序列会完全重复。
 - **不变量**：`余数 = r * 10 % denominator`。只要找到 `r` 重复出现，就找到循环节。
- **e) 最终正解的推导与核心代码结构**
 - **正解**：用 `map`（或数组）记录每个余数第一次出现的位置。从 `a % b` 开始模拟除法。当余数出现重复时，从第一次出现的位置到当前位置，就是循环节。如果余数为0，则说明是有限小数。
 - **核心代码**（伪代码）：

```
cpp int a, b; cin >> a >> b; string result = to_string(a / b) + "."; int remainder = a % b; map<int, int> pos; // 记录余数第一次出现的位置
string decimal_part = ""; int index = 0; while (remainder != 0) { if (pos.count(remainder)) { // 找到循环节 int start = pos[remainder]; string non_repeating = decimal_part.substr(0, start); string repeating = decimal_part.substr(start); result += non_repeating + "(" + repeating + ")"; break; } pos[remainder] = index; remainder *= 10; decimal_part += to_string(remainder / b); remainder %= b; index++; } if (remainder == 0) { // 有限小数 // 但如果之前已经用非循环节输出了，要小心处理 // 更简单的做法：在 while 循环外统一处理 } // 注意处理循环节为0的情况（即有限小数） // 输出结果
```
- **f) 推荐同类题**：
 - **P2613 【模板】有理数取余**：进一步练习有理数的模运算。
 - **P3858 [NOI2014] 随机数生成器** (只看第一部分)：让你理解随机数生成中“循环节”的思想。

题目 2：P1932 A+B A-B A*B A/B A%B Problem

- **a) AI题解摘要**：这是一道高精度综合题。不要用 `bc` 命令，你需要用一个字符串来实现大数的加、减、乘、除（取整）和取模运算。
- **b) 暴力思路怎么想？**

- **思路：**用 `int` 或 `long long` 直接计算。
- **复杂度：** $O(1)$ ，但会溢出。
- **部分分：**如果输入数字很大，直接爆零。
- **c) 瓶颈在哪里？**
 - **硬件：**语言内置的整数类型有上限 (10^{18} 左右)，无法处理任意大的数。
- **d) 关键性质/不变量观察 (Key Observation)**
 - **关键：**手工计算乘除法的核心思想：从低位向高位，逐位运算，并处理进位/借位。
 - **不变量：**过程中的进位/借位值。对于除法，每一次试商得到的余数，是下一轮的被除数的一部分。
- **e) 最终正解的推导与核心代码结构**
 - **正解：**将整数用 `vector<int>` (或 `string`) 存储，低位在前。实现高精度加法、减法（需注意比较大小，处理负数）、乘法、除法/取模。
 - **核心代码** (高精度加法示例)：

```
cpp vector<int> add(vector<int> a, vector<int> b)
{ // 确保 a 的位数 >= b 的位数 if (a.size() < b.size()) return add(b, a);
  vector<int> res; int carry = 0; for (int i = 0; i < a.size(); i++) { int sum =
  a[i] + carry; if (i < b.size()) sum += b[i]; res.push_back(sum % 10); carry =
  sum / 10; } if (carry) res.push_back(carry); return res; }
```
 - **高精度除法核心：**从高位开始，每次取一部分被除数，除以除数，得到一位商，余数保留给下一位。
- **f) 推荐同类题：**
 - **P1601 A+B Problem (高精)：**高精度入门。
 - **P1303 A*B Problem：**高精度乘法模板。

题目 3: B4071 [GESP202412 五级] 武器强化

- **a) AI题解摘要：**这是一道**贪心 + 模拟**题。重点是理解“强化失败不掉级”这个条件，并利用优先队列（最大堆或最小堆）来管理最小值。
- **b) 暴力思路怎么想？**
 - **思路：**枚举所有可能的强化顺序（全排列），计算总消耗，取最小值。
 - **复杂度：** $O(N!)$ 。当 $(N=5)$ 时还能算， $(N=100)$ 时就完全不可行。
 - **部分分：**能拿到“小数据”的部分分。
- **c) 瓶颈在哪里？**
 - **时间：**枚举所有顺序在 (N) 稍大时指数爆炸。

- d) 关键性质/不变量观察 (Key Observation)

- **关键**：题目要求“每次选取‘当前’强化等级最高的两个武器”，这不是一个固定顺序！强化会改变武器的等级。这是一个动态过程。
- **不变量**：强化材料的成本是固定的，你每次用掉的两个最低价值的武器，会合成一个更高级的武器。这本质上类似于**哈夫曼编码**的贪心思想：每次取最小的两个进行合并。

- e) 最终正解的推导与核心代码结构

- **正解**：用 `priority_queue<int, vector<int>, greater<int>>`（最小堆）来维护所有武器的“价值”。每次取出堆顶的两个最小元素（价值最低的），消耗材料，合并成一个新的武器（价值为两者的和 + 消耗的材料），再放回堆中。当堆中只剩一个武器时，结束。这次模拟的总消耗就是答案。
- **核心代码**：

```
```cpp #include using namespace std; int main() { int n, m; cin >> n >> m; priority_queue, greater\> pq; // 最小堆 for (int i = 1; i <= m; i++) { int p, c; cin >> p >> c; pq.push(c); // 价值入堆 }
```

`long long total_cost = 0; // 当堆中元素数量 > 1 时，始终进行合并 while (pq.size() > 1) { int a = pq.top(); pq.pop(); int b = pq.top(); pq.pop(); int new_value = a + b; // 合并后的新武器的基础价值？ // 注意：根据题意，消耗的材料就是 c，我们不需要把 c 加到总消耗里吗？ // 仔细看题：每次消耗一把对应材料，这把材料的价值是 c。 // 所以每次我们消耗的不是 a 和 b 的价值，而是 c。 // 因此，总消耗 = m 种材料的价值之和。因为我们进行了 m 次强化？ // 不是，我们每次只消耗一种材料，但材料种类有 m 种。 // 我们需要选一种方式使用材料，使得总消耗最小。 // 看懂题意：有 m 种材料，每种有 p 个，单个价值 c。 // 每次从所有武器中选两个等级最高的，然后选择一个材料进行强化，花费材料的价值。 // 所以问题是：我们怎么选择材料，使得总花费最少？ // 贪心：每次选两个等级最高的武器，然后用最便宜的可选材料强化。 // 但我们有优先队列维护武器等级？错！武器按等级排，但我们选“等级最高的”。 // 那我们就需要用最大堆来维护“当前等级最高的两个武器”。 // 每次取出堆顶的两个武器（等级最高），消耗当前可用材料中最便宜的一个，强化它们。 // 强化后，它们的等级+1，总价值 = 原价值之和 + 材料价值？ // 题面说：“强化后，被强化的两把武器将融为一体，变为等级+1的新武器，且其等级与两把武器之和相等。”这里的“有一定概率”？题面未提及概率，应是确定的。 // 其价值 = 原两武器价值之和 + 材料价值 c。 // 然后新武器入堆。 // 最终总花费 = 所有使用过的材料价值之和。 } }` 感觉用最大堆更符合直觉？继续思考。

**关键**：强化逻辑是：每次从**所有当前存在的武器**中，挑选**等级最高的两个**，然后选择**一种材料**进行强化，材料会消耗掉。强化的总花费=所用材料的价值总和。

所以武器等级是递减的？初始都是等级1。每次强化后，武器等级+1，所以新武器等级最高。因此堆中按等级排序，等级最高的在最上面。这是一个**最大堆**！

**最终结论:** - 用 `priority_queue` 存武器, 按**等级从大到小**排序。 - 用 `multiset` 或另一个 `priority_queue` 存材料, 按**价值从小到大**排序。 - 每次从武器堆顶取两个等级最高武器, 从材料堆顶取一个最便宜的材料, 进行强化。新武器等级=旧武器等级+1, 价值 = 旧武器价值之和+材料价值, 入武器堆。 - 材料消耗掉, 从材料堆中弹出。 - 重复直至只剩1个武器。 - 总花费 = 所有被使用的材料价值之和。

• **f) 推荐同类题:**

- **P1090 [NOIP2004 提高组] 合并果子:** 非常经典的“最小堆合并”问题, 也是这道题的核心思想来源!
- **P2161 [SHOI2009] 会场预约:** 对堆和优先级的综合性应用。

**题目 4: B3851 [GESP202306 四级] 图像压缩**

- **a) AI题解摘要:** 这是一道**模拟题**, 考验你对题目描述的遵循能力和位运算、数组的熟练度。没有高深算法, 但必须非常细心。
- **b) 暴力思路怎么想?**
  - **思路:** 严格按照题目给的5个步骤, 一步步用模拟实现。
  - **复杂度:** 步骤清晰, 主要复杂度在排序 ( $O(N \log N)$ ) 和查找上。
  - **部分分:** 如果步骤没理解错, 可以直接AC。
- **c) 瓶颈在哪里?**
  - **逻辑:** 不是性能瓶颈, 而是**理解题意**的瓶颈。尤其是对“统计出现次数、按频率排序、选取前16个作为颜色表、灰度压缩”这一系列流程的执行。
- **d) 关键性质/不变量观察 (Key Observation)**
  - **关键:** 题目描述就是最终答案。这是一个“需求文档”, 你要做的就是成为一台完美的翻译机, 把它翻译成代码。
  - **不变量:** `颜色表` 是固定的16色, 每种颜色用一个在 `[0, 15]` 的索引代表。
- **e) 最终正解的推导与核心代码结构**
  - **正解:**

读取图像数据, 建立颜色频率统计。频率高的优先。  
按频率从高到低排序, 如果频率相同按颜色值从小到大排序。  
选取频率最高的前16种颜色, 作为颜色表。如果不够16种, 就用颜色值小的补 (这题可能不需要, 但备着)。

遍历原始图像, 对每个像素点的颜色值, 在颜色表中找到最接近的颜色 (即差值绝对值最小), 并用颜色表中该颜色的索引代替。  
用四位二进制 (十六进制) 存储索引, 并输出压缩后的图像。

- **核心代码** (步骤2排序) : 

```
cpp // 假设我们有一个结构体 Color struct Color { int value; // 0-255 int freq; // 出现次数 }; vector<Color> colors; // ... 初始化 colors sort(colors.begin(), colors.end(), [](Color a, Color b) { if (a.freq != b.freq) return a.freq > b.freq; // 按频率降序 return a.value < b.value; // 频率相同按颜色值升序 });
```

- **f) 推荐同类题:**

- **P1540 [NOIP2010 提高组] 机器翻译**: 同样是模拟题, 考验对队列和模拟流程的理解。
- **P5582 [SWTR-01] Escape**: 一道流程稍微复杂的模拟题, 锻炼你翻译复杂题意的能力。

**选手, 你能走到这里, 说明你骨子里就是个搞竞赛的人。你缺的不是天赋, 而是“系统性”和“耐心”。这份报告是你的路书。接下来, 按照我给你的路线图, 一步一个脚印地走下去。记住: 真正的强者, 不是不犯错, 而是能从每一次错误中汲取养分, 然后走得更远。我在省选等你!**